“addressed” by a single index value, so it’s an order 1 tensor or 1-tensor for short. A matrix has two indices, one for each dimension, so it’s a 2-tensor. Higher order tensors can be referred to as a *k*-tensor, where *k* is the order, a non-negative integer. On the other end, a single number is a 0-tensor, also called a *scalar*, since it has no indices.

2.4.1. Automatic differentiation

The most important features of PyTorch that we need and that numpy doesn’t offer are automatic differentiation and optimization. Let’s say we want to set up a simple linear model to predict some data of interest. We can easily define the model using ordinary numpy-like syntax:

```
>>> x = torch.Tensor([2,4]) #input data
>>> m = torch.randn(2, requires_grad=True) #parameter 1
>>> b = torch.randn(1, requires_grad=True) #parameter 2
>>> y = m*x+b #linear model
>>> loss = (torch.sum(y_known - y))**2 #loss function
>>> loss.backward() #calculate gradients
>>> m.grad
tensor([  0.7734, -90.4993])
```

You simply supply the `requires_grad=True` argument to PyTorch tensors that you want to compute gradients for, and then call the `backward()` method on the last node in your computational graph, which will backpropagate gradients through all the nodes with `requires_grad=True`. You can then do gradient descent with the automatically computed gradients.

2.4.2. Building Models

For most of this book, we won’t bother dealing directly with automatically computed gradients. Instead we’ll use PyTorch’s `nn` module to easily set up a feedforward neural network model and then use the built-in optimization algorithms to automatically train the neural network without having to manually specify the mechanics of backpropagation and gradient descent. Here’s a simple two-layer neural network with an optimizer set up: